

ALLMÄNNA TIPS OM C++-PROGRAMMERING

v1.0

[Mattias Flodin](#) 2002-04-26

[Lura inte kompilatorn — undvik typkonvertering nedåt i klasshierarkin](#)
[Referenser, pekare? Vad är skillnaden?](#)
[Konsten att använda const](#)

Lura inte kompilatorn — undvik typkonvertering nedåt i klasshierarkin

Man, you look at this guy and think “Damn, this guy developed the C++ language. And he is still living, he is with us on earth. Now, right now.”

But then you look at his baldy head and think, “He sucks.” — Nagi Daou

Kompilatorn är din kompis. Om du är bra på att berätta för den vad du vill, så kan den berätta för dig när du gör saker du inte vill. Det är därför stark typkontroll, `const` och de många formerna av typkonvertering finns. Om inte kompilatorn har rätt går det förstås alltid att slå den på fingrarna och säga vad som *egentligen* gäller.

Låt mig ge ett exempel. I dessa tider när bankerna alltmer prioriterar “transaktionsintensiva kunder” börjar farmor Yster alltmer känna sig i skymundan. Var ska en pensionär gå när hon behöver spara pengar? På Förseningsbanken fnyser tjänstemännen föraktfullt åt henne var gång hon vill sätta in pengarna från den senaste trisslottvinsten. Därför har hon, företagsam som hon är, startat en egen bank för likasinnade. Till en början är det bara vännerna på bridgeklubben som sätter in pengar, och Yster håller reda på hur mycket pengar hennes kunder har med hjälp av en mycket välorganiserad pärm och en räknesticka.

Ryktet sprider sig dock snabbt! Snart vallfärdar pensionärer från grannsocknarna för att sätta in pengar, och även fattiga studenter som behöver spara till sommarhyran börjar göra små insättningar (som mot slutet av månaden brukar kulminera i ungefär lika stora uttag). All denna administration blir bara för mycket för Yster, så hon anställer dig för att skriva ett datorprogram som kan hålla reda på alla bankkonton. Du sätter dig ner och skriver en vettig basklass:

```
class BankAccount {
public:
    virtual ~BankAccount() = 0;
    virtual void deposit_money(double amount) = 0;
    virtual void withdraw_money(double amount) = 0;
    virtual double balance() const = 0;
    // ...
};
```

Som den gode STL-användare du är har du gjort en lista på alla konton på banken:

```
std::list<BankAccount*> all_accounts;
```

Bankens målgrupp är huvudsakligen snåla pensionärer, så det finns egentligen bara en sorts konto, ett sparkonto:

```
class SavingsAccount : public BankAccount {
public:
    // Lägger till månadens ränta till kontot
    void credit_interest();
    // ...
};
```

Pensionärerna kommer naturligtvis bli utom sig av raseri om de inte månatligen får ränta på sina pengar, så din första uppgift blir att skriva en funktion för att lägga till räntan till alla sparkonton. Du slänger genast ihop ett försök:

```
std::list<BankAccount*>::iterator it = all_accounts.begin();
while(it != all_accounts.end()) {
    (*it)->credit_interest();    // fel!!
    ++it;
}
```

varpå din kompis kompilatorn säger “stopp och belägg! BankAccount har ingen funktion som heter credit_interest!” Något frustrerande. Visst, all_accounts deklarerades att innehålla pekare till BankAccount, men du *vet* ju att den egentligen innehåller SavingsAccount. Som tur är kan du enkelt berätta detta för kompilatorn.

```
std::list<BankAccount*>::iterator it = all_accounts.begin();
while(it != all_accounts.end()) {
    static_cast<SavingsAccount*>(*it)->credit_interest();
    ++it;
}
```

Vad sa du nu då! Bossen visar var skåpet ska stå. Om kompilatorn är så dum att den inte förstår vad som finns i listan får man helt enkelt berätta det.

[Tryck här](#) för att höra hur det låter då [Bjarne](#) daskar till dig med en böckling.

Du har just tagit ett bra steg mot vad som på svenska kan kallas en “underhållsmardröm.” Och det handlar inte om cirkusclowner som sprutar vatten på dig hela natten tills du vaknar upp och inser att du kissat i sängen. Den sorts typomvandling som görs ovan heter på engelska *downcast*, d.v.s. en typomvandling nedåt i klasshierarkin. Det *fungerar* - för tillfället. Men sedan kommer revisorn.

Ysters system håller inte riktigt. Hon betalar ut räntor utan att egentligen tjäna några pengar, så snart hamnar hela företaget i konkurs. Förseningsbanken ser förstås en ypperlig möjlighet att få nya kunder, och lyckas med inte särskilt stora bekymmer köpa hela Ysters bank, inklusive dig. De vill nu lägga till checkkonton till arsenalen av kontotyper som Ysters bank har att erbjuda. Dessa konton är också räntebärande:

```
class CheckingAccount: public BankAccount {
public:
    void credit_interest();
    // ...
};
```

Plötsligt händer någonting farligt med ränterutinen. `all_accounts` kommer förstås ha både checkkonton och sparkonton, men räntefunktionen insisterar på att där bara finns sparkonton! Din kompis kompilatorn litar på dig i vått och torrt (vilket är nog så hjältemodigt med tanke på hur illa datorer klarar väta), så *programmet kompilerar fortfarande utan fel, men det fungerar inte*. Det är nu underhållsmardrömmen börjar visa sina rätta färger. Värre blir det om du gör det som Hin Håle frestande viskar i ditt öra, d.v.s. skriver programkod i stil med det här:

```
std::list<BankAccount*>::iterator it = all_accounts.begin();
while(it != all_accounts.end()) {
    if (*it pekar på ett SavingsAccount)
        static_cast<SavingsAccount*>(*it)->credit_interest();
    else
        static_cast<CheckingAccount*>(*it)->credit_interest();
    ++it;
}
```

Varje gång du kommer på dig själv med att skriva kod på formen “om objektet har typen T1 gör något, men om det är av typen T2, gör något annat” bör du se upp; Bjarne kan stå där och måtta med böcklingen. Sådan här kod kan man bli tvungen att skriva i C eller Pascal, men C++ kan bättre. Det är för just detta syfte som virtuella funktioner finns. Med virtuella funktioner får *kompilatorn* se till att rätt funktion anropas, beroende på den konkreta typen hos ett objekt. Då slipper du en massa skräp i koden som måste uppdateras varje gång en ny kontotyp blir till:

```
class BankAccount { /* ... */ };

// Ny basklass för konton som får ränta
class InterestBearingAccount: public BankAccount {
public:
    virtual void credit_interest() = 0;
    // ...
};
class SavingsAccount: public InterestBearingAccount {
    // ...
};
class CheckingAccount: public InterestBearingAccount {
    // ...
};
```

I god objektorienterad anda har vi tagit fasta på att sparkonton och checkkonton har något gemensamt: de är båda räntebärande konton. Nu kan ränterutinen skrivas om så att den ser ut så här.

```
std::list<BankAccount*>::iterator it = all_accounts.begin();
while(it != all_accounts.end()) {
    if (*it pekar på ett InterestBearingAccount)
        static_cast<InterestBearingAccount*>(*it)->credit_interest();
    ++it;
}
```

Det här är lite bättre – om vi lägger till fler konton som har ränta så ärver de alla från `InterestBearingAccount` och ränterutinen fungerar fortfarande, vilket kanske kan rädda oss från många ilska pensionärer. Men det är fortfarande inte bra. Vill vi klara dagen utan att stinka fisk behövs fler förändringar i designen.

Det ska erkännas att det här är ett *svårt* problem. Det finns inga perfekta lösningar som fungerar vid alla tillfällen. Men ett sätt är att begränsa vad kontolistan får innehålla. Om du istället kunde få en lista med `InterestBearingAccount` skulle problemet vara löst:

```
// Alla konton med ränta i banken
std::list<InterestBearingAccount*> all_ib_accounts;

std::list<BankAccount*>::iterator it = all_ib_accounts.begin();
while(it != all_accounts.end()) {
    (*it)->credit_interest();
    ++it;
}
```

Denna rutin skulle fungera inte bara nu, utan för alla framtida konton. Tyvärr har vi i någon mening flyttat problemet till ett annat ställe: Nu måste alla funktioner som skapar konton och lägger till dem till listan se till att alla relevanta listor uppdateras. Oftast är detta dock ett mindre problem. Hur man skapar ett objekt är av naturen beroende av exakt vilket konkret objekt vi vill ha, och samma funktion som har detta ansvar kan också få ansvar att se till att den läggs till de relevanta listorna. Man skulle till och med kunna tänka sig att konstruktorn ser till att lägga till sig själv till listan.

Om detta verkar skrämmande skulle du kunna låtsas som att problemet inte finns genom att säga "Alla konton har ränta, det är bara det att vissa har 0 % ränta." Du flyttar helt enkelt `credit_interest` till basklassen `BankAccount` och slipper alla typkonverteringar. I andra liknande fall kan man bli tvungen att skriva ut ett felmeddelande när en funktion anropas för "fel" datatyp. Resonemanget fungerar ibland, men leder ofta till filosofiska problem. Det är som att säga "Alla fåglar kan flyga. Pingvin är en fågel. Därför kan pingviner också flyga, men det är förbjudet för dem att försöka göra det."

Om du *måste* göra en downcast finns det bättre alternativ än `static_cast`. När måste du? Om `BankAccount` skrevs av självlärd Hacker-Hasse som specialarbete i gymnasiet, och allting finns i ett kompilerat bibliotek utan källkod, kan det vara omöjligt att ändra i klasshierarkin. Då kan du istället använda `dynamic_cast` för att göra konverteringen. Denna ger åtminstone en viss säkerhet att du inte gör en konvertering till något som du egentligen inte har. När du använder `dynamic_cast` på en pekare kommer en korrekt konverterad pekare returneras bara om omvandlingen lyckas. Misslyckas den får du en nollpekare som svar. Så här skulle det ursprungliga exemplet med "tryggare" typomvandling se ut:

```
std::list<BankAccount*>::iterator it = all_accounts.begin();
while(it != all_accounts.end()) {
    if (SavingsAccount* psa = dynamic_cast<SavingsAccount*>(*it))
        psa->credit_interest();
    else if (CheckingAccount* pca = dynamic_cast<CheckingAccount*>(*it))
        pca->credit_interest();
    else
        throw std::logic_error("Error! I don't recognize this account! :-(");
    ++it;
}
```

Referenser, pekare? Vad är skillnaden?

References are confusing in general. I used to think they were pointers, but they are

actually... like pointers in another dimension. — Jeff H.

I en mening är en referens ett alias för en variabel så att två namn kan syfta till samma objekt, medan en pekare är ett numeriskt värde som anger minnesadressen för ett objekt.

Ok, visst... så vad *är* skillnaden?

Rent maskinkodsmässigt är skillnaden praktiskt taget obefintlig; referenser och pekare är båda minnesadresser för processorn. Så vad ska referenser vara bra för? De uppfanns delvis för att C++ arbetar betydligt mer med objekt än C och det är kostsamt att skicka hela objekt “by value” eftersom de ofta är ganska stora. Med referenser blir det mindre bökigt än om vi måste skicka pekare hela tiden. Ta t.ex. funktionen

```
void print(string s)
{
    cout << s;
}
```

När `print(s)` anropas kommer strängen kopieras. Det görs automatiskt genom ett anrop till kopieringskonstruktorn för `string`, som måste allokera nytt minne för den textsträng som objektet äger och därefter kopiera tecken för tecken ur texten. Om funktionshuvudet istället är `void print(const string& s)` kommer en referens till den ursprungliga strängen skickas, men funktionsanropet ser precis likadant ut: `print(s)`. Att skicka en referens är betydligt snabbare än att kopiera hela objektet. Dessutom finns det vissa objekt som inte ens *går* att kopiera. Hade vi bytt till pekare så skulle kanske 350 anrop till `print` tvunget behöva ändras till `print(&s)`.

När bör referenser användas istället för pekare och tvärt om? Därom tvistar de lärde. En bra tumregel är att när objektet ska förändras av funktionen bör man skicka en pekare, men om objektet bara ska användas som “informationskälla” skickas en referens. Om vi till exempel istället hade funktionen `getline` som ska läsa en text från tangentbordet till en sträng kan det passa bättre med en pekare. Här är det viktigt att tänka på att använda `const` rätt; följer man den här principen så behöver man så gott som aldrig ha referensargument som *inte* är `const`, och på samma sätt behöver man inte ha pekarargument som *är* `const`.

En annan syn på skillnaden mellan referenser och pekare är att referenser *alltid* refererar till ett existerande objekt, medan pekare till exempel kan ha värdet noll. Därför skulle man då ha regeln att pekarargument bara används om det är tillåtet att utelämnas argumentet (genom att skicka en nollpekare). I alla andra fall skulle referenser användas. Problemet med detta är att koden riskerar att bli missförstådd. Ta t.ex. raderna

```
string s = "Hello World!";
foo(s);
```

Om `foo` tar en icke-`const` referens så skulle den mycket väl kunna ändra innehållet i strängen, vilket är svårt att gissa bara genom att titta på funktionsanropet. Om `foo` istället tar en pekare så blir programmeraren tvungen att skriva ett `&`-tecken framför `s` för att koden ska gå att kompilera. Tecknet blir som ett kontrakt mellan funktionen och den som anropar funktionen: “genom att sätta ett `&` här godkänner jag att du ändrar innehållet i variabeln.”

Så hur skriver man egentligen? Det finns ju &-tecken och *-tecken överallt, det är svårt nog att hålla isär dem för pekare. Ska man behöva kunna referenser nu också? Jepp, visst suger det. Det är priset vi får betala för att C++ ska vara bakåtkompatibelt med C. Inte nog med att vi måste leva med den kryptiska syntaxen för pekare, dessutom var de tvungna att använda samma tecken för referenser. Varför då? Därför att om nya tecken eller nyckelord skulle stoppas in i språket så skulle gamla C-program kanske inte gå att kompilera längre. Men du behöver inte vara deprimerad för det, tänk vad smart du kommer känna dig när du vet hur du ska använda dem!

En bra början för att hålla isär alla tecken är att lära sig hålla isär deklaration av en variabel och användning av en variabel. En deklaration är då man berättar för kompilatorn att variabeln finns. Vid en deklaration betyder * pekare och & referens.

Vid *användning* av en variabel behöver man inte bry sig ifall det är en referens. Pekare däremot behöver speciell behandling: & betyder "gör en pekare till det här objektet"; * betyder motsatsen, "gör ett objekt av den här pekaren." En referens *kan aldrig ändras efter den har skapats*. Det enda tillfället då tilldelningsoperatoren säger vad referensen ska referera till är under deklarationen. Därefter kommer den för evigt att bete sig precis som det objekt den refererar till.

```
int x=3;      // x är en heltalsvariabel med värdet 3
int y=4;      // y är en heltalsvariabel med värdet 4
int& z=x;     // z är en referens till ett heltal och refererar till x
z=y;         // Ändra värdet på z (och x!) till 4.
z=5;         // Ändra värdet på z och x till 5.
```

Motsvarande kod som använder pekare skulle se ut så här:

```
int x=3;      // x är en heltalsvariabel med värdet 3
int y=4;      // y är en heltalsvariabel med värdet 4
int* z=&x;    // z är en pekare till ett heltal och pekar till x
*z=y;        // Ändra värdet som z pekar på (dvs x) till 4.
*z=5;        // Ändra värdet som z pekar på till 5.
```

Varför ska man bry sig om allt det här? Varför inte bara låta argumenten kopieras istället för att krångla med referenser? Små saker som det här är vad som gör skillnaden i effektivitet mellan ett C-program och ett C++-program. Förhårdade C-programmerare älskar att klaga över att C++ bara är "syntaktiskt socker" som gör att deras program går långsammare. När du bemästrat referenser och flera andra trick kan du säga till dem att "om C++ gör att program går långsammare så beror det på programmeraren, inte på språket."

Som en fotnot finns det också tillfällen då man *måste* ha referenser, nämligen vid vissa operatoröverlagringar och framför allt för kopieringskonstruktorn. När du skickar ett objekt "by-value" (dvs inte som en referens) kommer den nya kopian att skapas genom att dess konstruktor anropas med det ursprungliga objektet som argument. Du skulle kunna skriva kopieringskonstruktorn så här:

```
class A {
public:
    // Vanlig konstruktor ("defaultkonstruktor")
    A(int x=0)
    {
```

```

        x_ = x;
    }
    // Kopieringskonstruktor
    A(A a)
    {
        x_ = a.x_;    // Kopiera värdet på x_ i a
    }
private:
    int x_;
};

```

Sedan anropar du funktionen `f(A a)` på följande sätt:

```

A a(25);
f(a);

```

Vad händer nu? Jo, kompilatorn försöker skapa en kopia av `a` genom att anropa kopieringskonstruktorn med `a` som argument. Men kopieringskonstruktorn är ju en funktion, den vill också ha en lokal kopia av `a`! För att lösa det anropas kopieringskonstruktorn igen, och igen, och igen...

Lösningen på detta är att låta kopieringskonstruktorn ta en referens som argument istället. Ingen extra kopiering behöver ske för att kunna anropa kopieringskonstruktorn, och allt är frid och fröjd.

Ett annat tillfälle då referenser är ett måste, är då man överlagrar tilldelningsoperatorn. Här måste operatorn returnera en referens till det egna objektet för att det ska vara möjligt med C-syntax som `a = b = c = d`.

Konsten att använda `const`

A co-worker just told me that they were looking for C++ programmers with no C++ experience but only programming experience. — Nagi Daou

Vad är felet med den här klassen?

```

class Pizza {
public:
    Pizza(int radius, int price) :
        radius_(radius),
        price_(price)
    {
    }
    int get_radius() {return radius_;}
    int get_price() {return price_;}
private:
    int radius_;
    int price_;
};

```

Den fungerar alldeles utmärkt ett tag, tills någon försöker skriva en funktion som tar en pizza som referens:

```
void f(const Pizza& pizza)
{
    cout << pizza.get_radius() << endl;
}
```

Problemet är att C++ utgår från att när `get_radius` anropas förändras objektets tillstånd. Men `pizza` är ju `const`! Kompilatorn vägrar kompilera.

Att ta bort `const` från funktionsparametern `pizza` löser problemet, men inte på ett bra sätt. Hela poängen är att vi vill garantera att `f()` inte får några sidoeffekter på parametern. Vad som behövs är ett sätt att berätta för kompilatorn att `get_radius` och `get_price` *inte* förändrar objektet. Detta görs helt enkelt genom att lägga till `const` i slutet på funktionshuvudet:

```
int get_radius() const {return radius_;}
int get_price() const {return price_;}
```

`const` är ett lätt sätt att undvika många buggar i program. Genom att förhindra att man “råkar” skriva över något kan hårbortfallet skjutas några år på framtiden. Många andra sätt finns, men just `const` är lite speciellt eftersom det inte går att använda det när man känner för det, och strunta i det på söndag morgon efter korridorsfesten. Om du glömmer `const` på funktionerna får det en *kedjeeffekt*. Inga andra funktioner kan använda `Pizza` utan att bryta mot reglerna. Intressanta koncept om du vill bygga upp en pizzamaffia, men inte så bra om du har funderingar på att göra ett program som fungerar.